Universitatea Politehnica din Timișoara

TITLE OF THE THESIS PAPER

Candidate: Given Name FAMILY NAME

Scientific coordinator: Professor/Associate professor/Assistant professor/Lecturer/Teaching assistant Given Name FAMILY NAME

Session: June 2023

CONTENTS

LIST OF FIGURES

LIST OF TABLES

LIST OF SNIPPETS

1. INTRODUCTION

1.1 APPROACHED DOMAINS

1.2 CONTEXT AND MOTIVATION

1.3 GENERAL SPECIFICATIONS

1.4 MAIN RESULTS

1.5 STRUCTURE OF THE PROJECT

The remainder of this work is organized as follows:

- **Chapter 2**
- **Chapter 3**
- **Chapter 4** is the core of this work. It documents the implementation of each software module: the lane keeping controller (Section 4.1), the maze solver (Section 4.2), the telemetry interface (Section 4.3), and the object-oriented architecture (Section 4.4).
- **Chapter ??**
- **Chapter 6**

2. STATE OF THE ART

2.1 PID CONTROL

2.2 FUZZY LOGIC AND FUZZY CONTROL

2.3 DEPTH-FIRST SEARCH FOR MAZE SOLVING

3. GENERAL DESCRIPTION

3.1 SYSTEM OVERVIEW

3.2 HARDWARE CONFIGURATION

3.3 SOFTWARE ARCHITECTURE

4. IMPLEMENTATION

4.1 THE LANE KEEPING MODULE

4.1.1 TASK DESCRIPTION

This project implements a robust algorithm in which the LEGO Mindstorms EV3 follows a trajectory on a white surface that is bordered with the help of two black lines between which the robot is situated, thus keeping itself in a lane.

Previous diploma projects including Mindstorms EV3 included the task of following a single black line. The purpose of the robot was to make correction turns when the sensor attached to it began to read another value other than the one of the black line. With the help of proportional, integral and derivative gains, the turns are adjusted from gentle turns to harsh ones.

So, unlike traditional robots that follow a single line this project focuses on a robot that uses two color sensors to balance itself inside the white “road” that is contrasted with the two black edges. It utilizes a custom controller combined with an emergency safety net to achieve a reliable and smooth navigation.

The robot is evaluating continuously its physical position by computing how far it has drifted from the ideal center and applying corrective steering to maintain its trajectory. In doing so, the robot uses two color sensors on each side of its front which adjust it from a disturbed state to an ideal state.

The Ideal State, described in the code by the variable setpoint, is the state in which the robot is perfectly centered, and both sensors read the exact reflection value from the white, meaning the difference is 0.

The Disturbed State is the state where the robot, by drifting left or right, moves over the black boundary line. This causes the sensor’s value on the black line to drop while the other sensor remains on white, so it keeps its value. In this situation, the difference between the two sensors becomes non-zero and it needs to be fed to control algorithms to be corrected.

The task configuration defines the following thresholds:

Listing 4.1: Lane keeping task configuration.

```
if task == "LaneKeeping":
    taskConfiguration={
        "LeftColorSensor":    Port.S2,
        "RightColorSensor":   Port.S3,
        "BlackThreshold":     6,
        "CrossingThreshold":  7,
        "MinDifference":      3,
        "BaseSpeed":          90,
        "TurningSpeed":       60,
        "EmergencySpeed":     30,
        "EmergencyTurnSpeed": 45,
```

```
        "StopConfirmCount":    1,
        "LogInterval":         1,
    }
```

The Left and Right Color Sensors are connected to the physical ports of the robot. The `BlackThreshold` represents value 6 where the sensors are above the black line, while `CrossingThreshold` represents the value 7 where the sensors are very close to the black line. The `MinDifference` being 3 represents the tolerance zone where the sensors are above white surface, but they don't read the exact same value (values close to 0 means dark and values close to 100 means bright). This allows the robot to continue straight, ignoring shadows on the white surface. The `BaseSpeed` is the forward velocity of the robot being perfectly centered. It is 90 mm/s since the track isn't long and can be easily followed. The `TurningSpeed` being 60 mm/s is the reduced forward velocity of the robot during a regulation to prevent the robot from overshooting its turns. The `EmergencySpeed` (30 mm/s) and `EmergencyTurnSpeed` (45 deg/s) are highly restrictive forward speed, respectively extreme rotational speed, used exclusively to aggressively pivot the robot away from a lane exit. The `StopConfirmCount` is the number of times the robot detects the value of black on both sensors before it stops. Due to false readings caused by shadows or oversteering in narrow corridors, this variable has been introduced. After many runs of the robot when there were no more false readings, the value of times it encounters black is set to 1. The `LogInterval` is the number of seconds to pass to print what the robot has recorded and what action did it took. It is 1 for a more detailed log.

4.1.2 THE LANE KEEPING CONTROLLER

The Lane Keeping application's functionality is described in the `LaneKeepingController` class, which implements the `Controller` interface. In this class there are defined the methods which contribute to the desired behavior of the robot when it tries to follow the lane. It firstly initializes the attributes in the constructor, retrieves the values from the sensors, prints the header of the log, decides what action needs to be taken and finally the run method which starts the robot to move.

Listing 4.2: LaneKeepingController constructor.

```
def __init__(self, algorithm, configuration, streamer, ev3, robot):
    super().__init__(algorithm, configuration, streamer, ev3, robot)
    self.leftSensor=ColorSensor(configuration["LeftColorSensor"])
    self.rightSensor=ColorSensor(configuration["RightColorSensor"])
    self.blackThreshold=configuration["BlackThreshold"]
    self.crossingThreshold=configuration["CrossingThreshold"]
    self.minDifference=configuration["MinDifference"]
    self.baseSpeed=configuration["BaseSpeed"]
    self.turningSpeed=configuration["TurningSpeed"]
    self.emergencySpeed=configuration["EmergencySpeed"]
    self.emergencyTurnSpeed=configuration["EmergencyTurnSpeed"]
    self.stopConfirmCount=configuration["StopConfirmCount"]
    self.logInterval=configuration["LogInterval"]
```

The constructor takes the inherited attributes from the super abstract class `Controller` and along with the more specific attributes taken from the `configuration` dictionary. These attributes initialize the state of the `LaneKeepingController` object.

Listing 4.3: LaneKeepingController sensors reading.

```
def readSensors(self):
    leftValue=self.leftSensor.reflection()
    rightValue=self.rightSensor.reflection()
    return leftValue, rightValue
```

This method simply reads the values of the light reflected on the surface they are currently on and returns them.

Listing 4.4: LaneKeepingController header printing.

```
def printHeader(self, algorithm):
    self.ev3.speaker.beep()
    print("\nLANE KEEPING-"+algorithm+"\n")
    print("Black Threshold:"+ str(self.blackThreshold))
    print("Crossing Threshold:"+ str(self.crossingThreshold))
    print("Min Difference:"+ str(self.minDifference))
    print("Base Speed:"+ str(self.baseSpeed))
    print("Turning Speed:"+ str(self.turningSpeed))
    print("Emergency Speed:"+ str(self.emergencySpeed)+"\n")
```

This method prints the header of the log to be able to compare the results and see how the thresholds affect the decision-making process after the run.

Listing 4.5: LaneKeepingController action deciding.

```
def decideAction(self, leftValue, rightValue, difference, dt):
    if leftValue<self.crossingThreshold and rightValue>=self.
        crossingThreshold:
         return self.emergencySpeed, self.emergencyTurnSpeed

    if rightValue<self.crossingThreshold and leftValue>=self.
        crossingThreshold:
         return self.emergencySpeed, -self.emergencyTurnSpeed

    if abs(disturbance)<=self.minDifference and isinstance(self.
        algorithm, (PIDAlgorithm, FuzzyPIAlgorithm)):
         return self.baseSpeed, 0

    disturbance, turnRate=self.algorithm.compute(0, disturbance, dt)

    return self.turningSpeed, turnRate
```

This method implements a safety layered architecture with three priority levels: emergency override, admitted tolerance and algorithm regulation. Based on this architecture, the method decides what action needs to be taken based on the values read by the sensors and the difference between them.

The first case treats the moment when the left sensor reads a value close to the black and the right sensor reads a larger value. This means that the robot is crossing the black line on the left side, so the method returns the emergency speed and the emergency turn speed with positive value, meaning turning right.

The second case is opposite to the first as it treats the moment when the right sensor reads a value close to the black and the left sensor reads a larger value. This means that the robot is crossing the black line on the right side, so the method returns the emergency speed and the emergency turn speed with negative value, meaning turning left.

The third case treats the difference with the values of the sensors being smaller or equal than the minimum difference accepted to let the robot continue straight ahead, meaning that the robot is on a bright surface and the method returns the normal speed and the speed of turning is 0 to move without deviation. This case is only applied for the PID and Fuzzy PI algorithms since the Fuzzy Rule-Based algorithm can have rules that apply even when the robot is close to the center.

In the fourth case the disturbance and turn rate are taken from the results done by the compute() method of the selected algorithm. The method finally returns the turning speed and the turn rate (the output of the regulation) meaning that the robot is regulated by the selected algorithm.

Listing 4.6: LaneKeepingController main run loop.

```
def run(self):
    self.printHeader(self.algorithmLabel())

    bothSensorsOnLineCount=0
    lastTime=0.0

    try:
        while True:
            step, currentTime=self.nextStep()
            dt=currentTime-lastTime
            lastTime=currentTime

            leftValue, rightValue=self.readSensors()

            leftOnBlack=leftValue<self.blackThreshold
            rightOnBlack=rightValue<self.blackThreshold

            if leftOnBlack and rightOnBlack:
                bothSensorsOnLineCount+=1
                if bothSensorsOnLineCount>=self.stopConfirmCount:
                    self.robot.stop()
                    self.ev3.speaker.beep()
                    print("\nBoth sensors on line for "+str(self.
                        stopConfirmCount)+" consecutive steps. Stopping."
                        )
                    break

```

```
26                  self.robot.drive(10, 0)
27                  wait(20)
28                  continue
29              else:
30                  bothSensorsOnLineCount=0
31
32              disturbance=leftValue-rightValue
33              speed, turnRate=self.decideAction(leftValue, rightValue,
                    disturbance, dt)
34              self.streamer.send(self.step, disturbance, turnRate)
35              self.logger.log(self.step, disturbance, turnRate)
36
37              if step%self.logInterval==0:
38                  print("Step "+str(step)+" Left: "+str(leftValue)+" Right
                        : "+str(rightValue))
39                  if isinstance(self.algorithm, FuzzyRuleBased):
40                      print(self.algorithm.logRules())
41
42              if speed<30:
43                  speed=30
44
45              self.robot.drive(speed, turnRate)
46              wait(20)
47
48      except KeyboardInterrupt:
49          self.robot.stop()
50          self.ev3.speaker.beep()
51          print("\nLANE KEEPING STOPPED\n")
52      finally:
53          self.streamer.close()
54          self.logger.close()
```

The final run() method is the main execution loop. It combines four phases of the control cycle: sense, decide, actuate and wait. It includes the previous three methods to execute the task. Firstly, it prints the header according to the algorithm returned by the `algorithmLabel()` method where its instance was checked and the constants and resets the number of times both sensors crossed the black line and the time.

The continuous loop begins with the computation of the change in time by subtracting the current time (obtained from the `now()` method) with the last time and updating the last time. The left and right values are assigned with the values read by the sensors and then, based on the log interval, which in this case is 1, is printed at each multiple of it the step plus what values the sensors read. If the algorithm is Fuzzy Rule Based, the rules are also printed.

With the help of the Boolean variables `leftOnBlack` and `rightOnBlack`, the condition for the robot to stop is created by having both values of the sensors smaller than the threshold value which indicates a very dark surface. In the event of false readings, the number of times both sensors were on the black line increases until it reaches the confirmation count to stop, otherwise it reduces the speed and waits to see if indeed is on a black line or not.

If the conditions fail, the disturbance is computed and the speed with the turn rate are given by the action it needs to be decided based on the left value, right value, disturbance and

the change in time. The returned speed can't get lower than 10 and the robot is driving based on the final speed and turn rate. The final wait is used to ensure that the loop frequency remains consistent regardless of which branch was taken.

The except part of the try-except black where the program is interrupted by the user. The robot stops and the stream with the file are closed.

4.1.3 PID ALGORITHM

One possible option for the robot to succeed in following the lane is with the help of a PID controller. This algorithm that implements this type of controller uses Comparative Control, where the disturbance of the two color sensors, is computed at each step in order to be given as input to the controller so that it can generate the turn rate so that the disturbance is minimized.

Listing 4.7: PIDAlgorithm constructor.

```
def __init__(self, configuration):
    self.kp=configuration["Kp"]
    self.ki=configuration["Ki"]
    self.kd=configuration["Kd"]
    self.minOutput=configuration["MinOutput"]
    self.maxOutput=configuration["MaxOutput"]
    self.integralMin=configuration["IntegralMin"]
    self.integralMax=configuration["IntegralMax"]
    self.integral=0.0
    self.previousDisturbance=0.0
```

In the first part of the definition of the class, the constructor initializes the constants which will be computed to determine the final turn rate, as well as the interval ends for it and the integral. The previous disturbance and the integral are set to 0.

Listing 4.8: PIDAlgorithm compute method.

```
def compute(self, setPoint, measuredPoint, dt):
    disturbance=setPoint-measuredPoint
    pTerm=self.kp*disturbance

    if dt>0:
        self.integral+=disturbance*dt

        if self.integral>self.integralMax:
            self.integral=self.integralMax
        elif self.integral<self.integralMin:
            self.integral=self.integralMin
    iTerm=self.ki*self.integral

    if dt>0:
        derivative=(disturbance-self.previousDisturbance)/dt
    else:
        derivative=0.0
```

```
18
19      dTerm=self.kd*derivative
20
21      output=pTerm+iTerm+dTerm
22
23      if output>self.maxOutput:
24          output=self.maxOutput
25      elif output<self.minOutput:
26          output=self.minOutput
27
28      self.previousDisturbance=disturbance
29
30      return disturbance, output
```

The `compute()` method takes the `setPoint`, which is usually 0 indicating perfect balance between sensors, the `measuredPoint` which is the actual difference between sensors, and the change in time. The disturbance is the result of ideal minus actual. The final turn rate output is the sum of the three PID terms made of the imbalance and their gains:

$$\text{Turn Rate} = \left(K_P \times \text{Imbalance}\right) + \left(K_I \times \int \text{Imbalance}\, dt\right) + \left(K_D \times \frac{d(\text{Imbalance})}{dt}\right) \quad (4.1)$$

The Derivative Gain symbolizes the present as it only considers the current value of the Disturbance. As the main steering force, its effect on the robot is that the further the robot is from the center, the harder it turns. In other words, if the robot is sluggish and goes off the track on curves, increase the K_P. If the robot shakes violently back and forth, decrease the K_P. In the `compute()` method the proportional term is the product of the disturbance and the gain.

The Integral Gain represents the memory since it accumulates the Disturbance over time. If the robot has a mechanical drift, for example a slightly weaker motor, or is on a long, sweeping curve, K_P alone might leave the robot off-center while K_I builds slowly steering power until the robot is perfectly centered. Usually this constant is very small, less than 0.1. Otherwise, it creates Integral Windup, meaning that the robot will overreact drastically and swing off the lane. In the `compute()` method the integral term is the product of gain and the disturbance multiplied with the change in time, constrained by the integral interval in order to prevent the integral from growing infinitely large.

The Derivative Gain is the dampener as it computes how fast the Disturbance is changing. When the robot steers back toward the center of the lane, K_D acts as a brake. It reduces the steering power right before the robot reaches the center, preventing it from overshooting into the other black line. A higher K_D will tighten the path so that the robot will avoid crawling widely as a snake down the lane. In the `compute()` method the derivative term is the product of the gain and the derivative of the disturbance with respect to the time.

The output final turn rate is comprised of all three PID terms and restricted to the output interval so that the motors of the robot don't receive impossible commands.

Listing 4.9: PIDAlgorithm reset method.

```
def reset(self):
        self.integral=0.0
        self.previousDisturbance=0.0
```

The previous disturbance is updated with the current one and the method returns the disturbance and the output. The PIDAlgorithm class also has a reset method to clear the memory of the controller by setting the integral and the previous disturbance to 0.

4.1.4 FUZZY PI ALGORITHM

This algorithm stands apart from a rigid mathematical model as the PID controller by acting as a human driver who adjusts the steering power based on how off the center the robot is. A Fuzzy PI controller changes the constants K_P and K_I depending on the state of the robot using Fuzzy Logic.

In standard programming the logic is binary, where conditions are either true (1) or false (0). Fuzzy Logic introduces degrees of truth, allowing the robot to categorize a disturbance slightly off-center or very far off-center. By translating the crisp value of the Disturbance into fuzzy sets, the algorithm adjusts dynamically the proportional and integral gains to generate the final turn rate.

Listing 4.10: FuzzyPIAlgorithm constructor.

```
def __init__(self, configuration):
    self.baseKp=configuration["BaseKP"]
    self.baseKi=configuration["BaseKI"]
    self.smallDisturbance=configuration["SmallDisturbance"]
    self.mediumDisturbance=configuration["MediumDisturbance"]
    self.largeDisturbance=configuration["LargeDisturbance"]
    self.fuzzyMinOutput=configuration["FuzzyMinOutput"]
    self.fuzzyMaxOutput=configuration["FuzzyMaxOutput"]
    self.fuzzyIntegralMin=configuration["FuzzyIntegralMin"]
    self.fuzzyIntegralMax=configuration["FuzzyIntegralMax"]
    self.rules=configuration["Rules"]
    self.integral=0.0
```

The `FuzzyPIAlgorithm` class implements the fuzzy PI algorithm following the three stages of a fuzzy control system: fuzzification, fuzzy inference and defuzzification. The fuzzification is the conversion of a crisp numerical value into a fuzzy set. The fuzzy inference is the approximate reasoning to formulate a mapping from premise to conclusion. The defuzzification is the reverse conversion of a fuzzy set into a crisp value. The constructor of the fuzzy PI algorithm initializes the base gains K_P and K_I which will be computed to determine the final turn rate, the fuzzy set boundaries S=3, M=6, L=10 describing the disturbance, the rule tuples that encode the gain multipliers for each fuzzy set.

Fuzzification

Listing 4.11: FuzzyPIAlgorithm fuzzification.

```
def fuzzifyDisturbance(self, disturbance):
    disturbanceAbs=abs(disturbance)

    if disturbanceAbs<=self.smallDisturbance:
        muSmall=1.0
    elif disturbanceAbs<=self.mediumDisturbance:
        muSmall=(self.mediumDisturbance-disturbanceAbs)/(self.
            mediumDisturbance-self.smallDisturbance)
    else:
        muSmall=0.0

    if disturbanceAbs<=self.smallDisturbance:
        muMedium=0.0
    elif disturbanceAbs<=self.mediumDisturbance:
        muMedium=(disturbanceAbs-self.smallDisturbance)/(self.
            mediumDisturbance-self.smallDisturbance)
    elif disturbanceAbs<=self.largeDisturbance:
        muMedium=(self.largeDisturbance-disturbanceAbs)/(self.
            largeDisturbance-self.mediumDisturbance)
    else:
        muMedium=0.0

    if disturbanceAbs<=self.mediumDisturbance:
        muLarge=0.0
    elif disturbanceAbs<=self.largeDisturbance:
        muLarge=(disturbanceAbs-self.mediumDisturbance)/(self.
            largeDisturbance-self.mediumDisturbance)
    else:
        muLarge=1.0

    return muSmall, muMedium, muLarge
```

The `fuzzifyDisturbance()` method takes the crisp value of the disturbance and computes the membership degree for each fuzzy set: small S, medium M and large L. The μ_S, μ_M, μ_L, representing the membership degrees, are values between 0 and 1 and describe how much the input resembles each set simultaneously.

The three fuzzy sets are defined by triangular and trapezoidal membership functions. They are delimited by the boundary values from the configuration: S=3.0, M=6.0 and L=10.0. These boundaries determine where one fuzzy set begins and another ends, with overlapping regions where the disturbance partially belongs to two sets at the same time. It checks for each fuzzy set because there can be more than one non-zero membership value, and that's why the method returns all three membership values.

The picture above presents graphically the intervals where the disturbance can be situated and what membership value can have, describing a triangular/trapezoidal transition. The disturbance grows or is reduced resulting in smaller membership function for the initial set and a higher membership function for the next set.

For *Small*, a trapezoidal membership function, has full membership ($\mu_S = 1.0$) when $|d| \leq S$. It then decreases linearly from 1 to 0 as the disturbance increases from S to M. Zero membership ($\mu_S = 0.0$) when $|d| \geq M$.

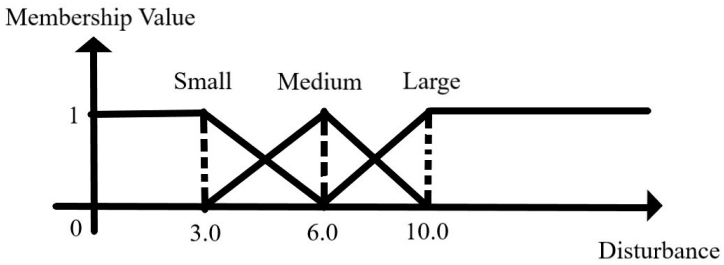

Figure 4.1: Fuzzy membership functions for Small, Medium and Large disturbance sets with boundary values $S = 3.0$, $M = 6.0$ and $L = 10.0$.

For *Medium*, a triangular membership function, has zero membership ($\mu_M = 0.0$) when $|d| \leq S$ or $|d| \geq L$. It increases linearly from 0 to 1 as the disturbance increases from S to M until it reaches its peak when the disturbance is exactly M and has full membership function ($\mu_M = 1.0$). Then it decreases linearly from 1 to 0 as the disturbance increases from M to L.

For *Large*, a trapezoidal membership function, has zero membership ($\mu_L = 0.0$) when $|d| \leq M$. Then it increases linearly from 0 to 1 as the disturbance increases from M to L and reaches full membership ($\mu_L = 1.0$) for any disturbance greater or equal to L.

These membership functions have the property that at any value of the disturbance the sum of membership values is 1. This ensures a smooth transition between the fuzzy sets and better control for the robot's movement.

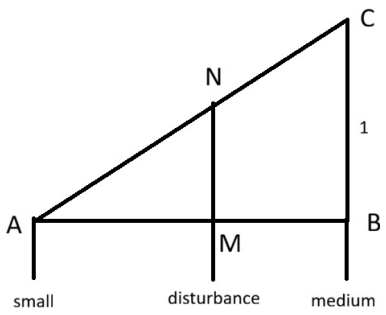

Figure 4.2: Slope of Membership Function for the Medium Fuzzy Set with a disturbance value between Small and Medium.

This picture represents the ascending slope of the medium membership function and

the disturbance having a value between small and large. It creates two similar right triangles, ABC which is formed by the full slope and AMN which is the smaller triangle formed by the membership value of the disturbance.

This means that MN/BC is equal to AM/AB, so the side $MN = AM \times BC/AB$. Since BC is equal to 1, AM is disturbance $-$ small and AB is medium $-$ small, the membership degree of the disturbance (MN) is $(\text{disturbance} - \text{small})/(\text{medium} - \text{small})$. This geometric reasoning is applied to every ascending and descending slope for each membership function in order to compute every membership value.

Fuzzy Inference Rules & Defuzzification

Listing 4.12: FuzzyPIAlgorithm inference & defuzzification.

```
def inference(self, muSmall, muMedium, muLarge):
    kpSmall = self.rules[0][0]
    kiSmall = self.rules[0][1]

    kpMedium = self.rules[1][0]
    kiMedium = self.rules[1][1]

    kpLarge = self.rules[2][0]
    kiLarge = self.rules[2][1]

    totalWeight = muSmall + muMedium + muLarge

    if totalWeight > 0:
        kpMultiplier = (muSmall * kpSmall + muMedium * kpMedium +
            muLarge * kpLarge) / totalWeight
        kiMultiplier = (muSmall * kiSmall + muMedium * kiMedium +
            muLarge * kiLarge) / totalWeight
    else:
        kpMultiplier = 1.0
        kiMultiplier = 1.0

    kpAdaptive = self.baseKp * kpMultiplier
    kiAdaptive = self.baseKi * kiMultiplier

    return kpAdaptive, kiAdaptive
```

The inference stage applies a set of IF-THEN rules that map each fuzzy set to specific gain multiplier values. These rules encode the control strategy: how aggressively the controller should respond to the magnitude of the disturbance. The `inference()` method handles the inference and defuzzification stages of the fuzzy control system. The Fuzzy PI algorithm uses three rules, stored in the configuration (the list Rules: $[(0.8, 0.7), (1.0, 1.0), (1.2, 1.3)]$) where each tuple represents the K_P and K_I multiplier for the corresponding fuzzy set.

Rule 1: IF the disturbance is small THEN $K_P \times 0.8$ and $K_I \times 0.7$. This is interpreted as: If the disturbance between the values read by the sensors is small and the robot is near

the center then the gains are reduced, K_P by 20% and K_I by 30%, for smooth and gentle corrections. This prevents the robot from making oscillations around the center position.

Rule 2: IF the disturbance is medium THEN $K_P \times 1.0$ and $K_I \times 1.0$. This is interpreted as: If the disturbance is medium and the robot is moderately off-center then keep the nominal gains. This is the nominal point of the controller.

Rule 3: IF the disturbance is large THEN $K_P \times 1.2$ and $K_I \times 1.3$. This is interpreted as: If the disturbance is large and the robot is far off-center then boost the gains, K_P by 20

The defuzzification stage is the conversion of the fuzzy set back into a crisp value. The method mentioned previously computes a center of mass formula for defuzzification. From the inference stage, the controller combines the gain multipliers by aggregating all active rules into a pair of final adaptive gains. The denominator, being the sum of all membership values, is always equal to 1 and ensures that the result is normalized. These adaptive gains are then given to the `compute()` method to obtain the final turn rate.

Computing the Output

Listing 4.13: FuzzyPIAlgorithm output computing

```
def compute(self, setPoint, measuredPoint, dt):
    disturbance=setPoint-measuredPoint
    muSmall, muMedium, muLarge=self.fuzzifyDisturbance(disturbance)
    kpAdaptive, kiAdaptive=self.inference(muSmall, muMedium, muLarge)

    pTerm=kpAdaptive*disturbance

    if dt>0:
        self.integral+=disturbance*dt

        if self.integral>self.fuzzyIntegralMax:
            self.integral=self.fuzzyIntegralMax
        elif self.integral<self.fuzzyIntegralMin:
            self.integral=self.fuzzyIntegralMin

    iTerm=kiAdaptive*self.integral

    output=pTerm+iTerm

    if output>self.fuzzyMaxOutput:
        output=self.fuzzyMaxOutput
    elif output<self.fuzzyMinOutput:
        output=self.fuzzyMinOutput

    return disturbance, output
```

The `compute()` method combines all stages of the fuzzy logic to obtain the final turn rate the robot must execute. It receives the `setPoint` which is usually 0, indicating perfect balance between sensors, the `measuredPoint` which is the actual difference between sensors and the change in time. The disturbance is the difference between `setPoint` and

`measuredPoint`. Using the adaptive K_P and K_I computed from the fuzzy system, there are constructed the PI terms of the controller.

The proportional term is the product of the disturbance and the adaptive gain and acts as the correction to the current disturbance, while the integral term is the product of gain and the disturbance multiplied with the change in time, constrained by the integral interval in order to prevent the integral from growing infinitely large.

$$Output = (K_P \times Imbalance) + (K_I \times \int Imbalance\ dt) \tag{4.2}$$

The output final turn rate is comprised of the two PI terms and restricted to the output interval, so that the motors of the robot don't receive impossible commands. The method returns the disturbance and the output representing the turn rate.

The `reset()` method is used to clear the integral accumulator, and it's called whenever the controller is reinitialized. For example, after the robot handles a lane crossing event.

The advantage of this approach over a fixed-gain PID controller is that the Fuzzy PI controller automatically adjusts its output according to its position from the center. It weakens when the robot is close to the center to avoid jitters and strengthens the output when the robot is far from the center to avoid sluggish reaction to large deviations. In this behavior, the fuzzy logic provides a smooth transition between control strategies.

4.1.5 FUZZY RULE-BASED ALGORITHM

This algorithm relies only on fuzzy logic to control the movement of the robot based on disturbance created by the difference between values read from the sensors. Unlike the Fuzzy PI algorithm, which only tunes the gains of the PI controller, the value of disturbance and the derivative of disturbance are fuzzified and mapped through a complete Mamdani type fuzzy inference system directly to the turn rate given to the motor.

This approach has two advantages. The first one is the control strategy is expressed in natural language rules rather than mathematical gain values. This makes it easier to tune the boundaries of the fuzzy sets. The second one is by using the current disturbance and the rate of change (trend) the controller is able to anticipate future states. The correction is weakened when the robot is recovering to the center and it is strengthened when the robot begins to deviate from the center.

The algorithm is implemented in the `FuzzyRuleBased` class and comprises of the four stages of the Mamdani fuzzy controller: fuzzification of the disturbance and the trend, fuzzy inference through a set of 15 rules, aggregation of the active conclusion rules and defuzzification to obtain the crisp final turn rate.

Listing 4.14: FuzzyRuleBasedAlgorithm constructor

```
1 class FuzzyRuleBased(Algorithm):
2     def __init__(self, configuration):
3         self.smallDisturbance=configuration["SmallDisturbance"]
4         self.mediumDisturbance=configuration["MediumDisturbance"]
```

```
        self.deltaThreshold=configuration["DeltaThreshold"]
        self.smallOutput=configuration["SmallOutput"]
        self.mediumOutput=configuration["MediumOutput"]
        self.largeOutput=configuration["LargeOutput"]
        self.rulesTable=configuration["Rules"]

        self.previousDisturbance=0.0
        self.firstCall = True
        self.lastActiveRules=[]

        self.outputSingletons=[
            -self.largeOutput,
            -self.mediumOutput,
            -self.smallOutput,
            0.0,
            self.smallOutput,
            self.mediumOutput,
            self.largeOutput,
        ]
```

The `FuzzyRuleBased` begins with the constructor, initializing from the configuration the fuzzy sets for disturbance, derivative of disturbance and the output, the interval ends restricting the output and the fuzzy rules. The interval ends are necessary to prevent the motor of the robot from executing impossible commands. The `previousDisturbance` is set to 0 to compute the derivative. The variables `firstCall`, `lastActiveRules` and `previousActiveRules` keep track of which rules are firing and by how much to debug easier. The output singletons define the magnitude of the turn rate.

Listing 4.15: FuzzyRuleBasedAlgorithm clamp method

```
def clamp(self, value):
        if value<0:
            return 0.0
        elif value>1:
            return 1.0
        return value
```

The `clamp()` method is a helper function which limits the range of the membership value of each fuzzy set to be between 0 and 1.

Fuzzification of Disturbance

Listing 4.16: FuzzyRuleBasedAlgorithm disturbance fuzzyfication

```
def fuzzifyDisturbance(self, disturbance):
    S=self.smallDisturbance
    M=self.mediumDisturbance

    muNL=self.clamp((-S-disturbance)/(M-S))
```

```
6
7       if disturbance<=-M or disturbance>=0.0:
8           muNS=0.0
9       elif disturbance<=-S:
10          muNS=(disturbance+M)/(M-S)
11      else:
12          muNS=(-disturbance)/S
13
14      if disturbance<=-S or disturbance>=S:
15          muZE=0.0
16      elif disturbance<0.0:
17          muZE=(disturbance+S)/S
18      else:
19          muZE=(S-disturbance)/S
20
21      if disturbance<=0.0 or disturbance>=M:
22          muPS=0.0
23      elif disturbance<=S:
24          muPS=disturbance/S
25      else:
26          muPS=(M-disturbance)/(M-S)
27
28      muPL=self.clamp((disturbance-S)/(M-S))
29
30      return (muNL, muNS, muZE, muPS, muPL)
```

The `fuzzifyDisturbance()` method takes the crisp value of the disturbance, as the first input, and computes the membership degree for each fuzzy set: negative large NL, negative small NS, zero ZE, positive small PS and positive large PL. The μNL, μNS, μZE, μPS and μPL represents the membership degrees, with values between 0 and 1 and describe how much the input resembles each set simultaneously. The reason these fuzzy sets are defined on signed disturbances, unlike the Fuzzy PI algorithm, is that this allows the controller to distinguish between left and right deviation.

These five fuzzy sets are defined by triangular and trapezoidal membership functions. They are delimited by the boundary and signed values from the configuration: -S=-3.0, -M=-6.0, +S=3.0, +M=6.0. These boundaries determine where one fuzzy set begins and another ends, with overlapping regions where the disturbance partially belongs to two sets at the same time. It checks for each fuzzy set because there can be more than one non-zero membership value, and that's why the method returns all five membership values.

The picture above presents graphically the intervals where the disturbance can be situated and what membership value can have, describing a triangular/trapezoidal transition. The disturbance grows or is reduced resulting in smaller membership function for the initial set and a higher membership function for the next set.

For negative large, a trapezoidal membership function, has full membership (μNL = 1.0) when $d \leq -M$. Then it decreases linearly from 1 to 0 as the disturbance increases from -M to -S. Zero membership (μNL = 0.0) when $d > -S$. This represents a strong drift to the left.

For negative small, a triangular membership function, has zero membership (μNS = 0.0) when $d < -M$ or $d > 0$. It increases linearly from 0 to 1 as the disturbance increases from

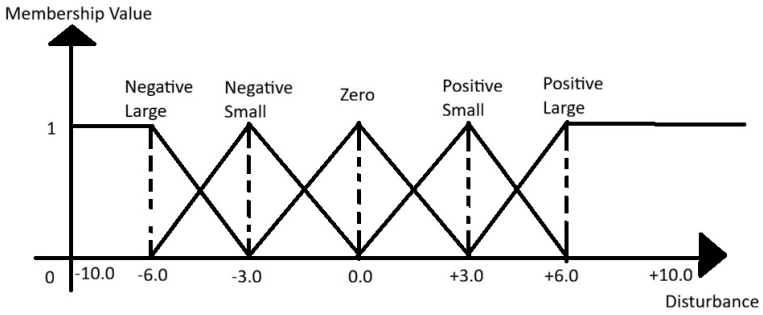

Figure 4.3: Fuzzy membership functions for Small, Medium and Large disturbance sets with boundary values $S = \pm 3.0$, $M = \pm 6.0$ and $L = \pm 10.0$.

-M to -S until it reaches its peak when the disturbance is exactly -S and has full membership (μNS = 1.0). Then it decreases linearly from 1 to 0 as the disturbance increases from -S to 0. This represents a slight drift to the left.

For zero, a triangular membership function, has zero membership (μZE = 0.0) when $d < -S$ or $d > +S$. It increases linearly from 0 to 1 as the disturbance increases from -S to 0 until it reaches its peak when the disturbance is exactly 0 and has full membership (μZE = 1.0). Then it decreases linearly from 1 to 0 as the disturbance increases from 0 to +S. This represents the robot being approximately centered in the lane.

For positive small, a triangular membership function, has zero membership (μPS = 0.0) when $d < 0$ or $d > +M$. It increases linearly from 0 to 1 as the disturbance increases from 0 to +S until it reaches its peak when the disturbance is exactly +S and has full membership (μPS = 1.0). Then it decreases linearly from 1 to 0 as the disturbance increases from +S to +M. This represents a slight drift to the right.

For positive large, a trapezoidal membership function, has full membership (μPL = 1.0) when $d \geq M$. It decreases linearly from 1 to 0 as the disturbance decreases from +M to +S. Zero membership (μPL = 0.0) when $d < +M$.

This picture represents the ascending slope of the medium membership function and the disturbance having a value between small and large. It creates two similar right triangles, ABC which is formed by the full slope and AMN which is the smaller triangle formed by the membership value of the disturbance.

This means that MN/BC is equal to AM/AB, so the side $MN = AM \times BC/AB$. Since BC is equal to 1, AM is $disturbance - small$ and AB is $medium - small$, the membership degree of the disturbance (MN) is $(disturbance\ -\ small)/(medium\ -\ small)$. This geometric reasoning is applied to every ascending and descending slope for each membership function in order to compute every membership value, as was proved for the Fuzzy PI algorithm. The trapezoidal functions have only one slope and the membership values are implemented in the

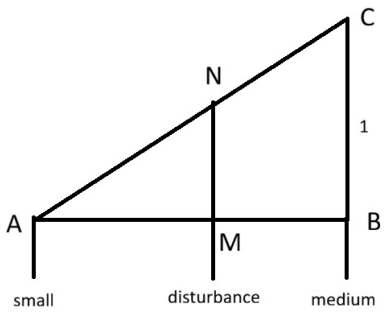

Figure 4.4: Slope of Membership Function for the Medium Fuzzy Set with a disturbance value between Small and Medium.

code with the help from the `clamp()` method to restrict the results to the interval [0, 1], while the triangular functions have two mirrored slopes.

Fuzzification of Trend

The second input for the fuzzy control system is how the disturbance is changing between consecutive control cycles. It is computed as $\Delta d = d_{\text{current}} - d_{\text{previous}}$. The physical interpretation of the trend depends on its sign. A positive trend means the new disturbance is higher than the previous one, so the robot is moving further from the center while a negative trend means the new disturbance is lower than the previous one, so the robot is recovering towards the center.

Listing 4.17: FuzzyRuleBasedAlgorithm trend fuzzyfication

```
def fuzzifyDelta(self, delta):
    D=self.deltaThreshold

    muN=self.clamp((-delta)/D)

    if delta<=-D or delta>=D:
        muZ=0.0
    elif delta<0.0:
        muZ=(delta+D)/D
    else:
        muZ=(D-delta)/D

    muP=self.clamp(delta/D)

    return (muN, muZ, muP)
```

The `fuzzifyDelta()` method takes the crisp value of trend of disturbance. It computes the membership degree for each fuzzy set: negative N, zero Z, and positive P. The μN, μZ and μP represents the membership degrees, with values between 0 and 1 and describe how much the input resembles each set simultaneously.

These three fuzzy sets are defined by triangular and trapezoidal membership functions. They are delimited by the threshold value from the configuration: D=3.0.

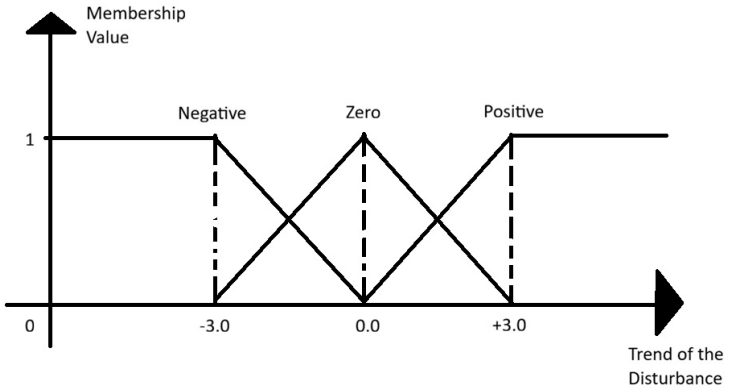

Figure 4.5: Fuzzy membership functions for Negative, Zero and Positive trend of disturbance sets with boundary value $D = \pm 3.0$.

The picture above presents graphically the intervals where the trend of disturbance can be situated and what membership value can have, describing a triangular/trapezoidal transition.

For negative, a trapezoidal function, has full membership when $\Delta d \leq -D$ and decreases linearly to 0 at $\Delta d = 0$. This represents the fact that the disturbance tends to decline and the robot recovers to the center.

For zero, a triangular function, has full membership at the peak $\Delta d = 0$ and decreases linearly to the base from $-D$ to $+D$. This represents a stable disturbance with little change.

For positive, a trapezoidal function, has full membership when $\Delta d \geq +D$ and decreases linearly to 0 at $\Delta d = 0$. This represents the fact that the disturbance tends to grow and the robot furthers away from the center.

Fuzzy Inference Rules

The core of the Fuzzy Rule-Based controller is a set of 15 IF-THEN rules (five fuzzy sets for the disturbance and three for the trend) that map every combination of disturbance and trend to a final output turn rate. The output is expressed as seven fuzzy sets (negative large NL = -50, negative medium NM, negative NS, zero ZE, positive small PS, positive medium PM and positive large PL). Each set is associated with a singleton value that represents the magnitude of turn rate in degrees per second in the table below:

Table 4.1: Fuzzy Rule-Based: output labels and their crisp turn rate values.

NL	NM	NS	ZE	PS	PM	PL
-50	-30	-10	0	$+10$	$+30$	$+50$

Negative output values command the robot to turn left, positive values command the robot to turn right and zero commands the robot to continue straight. The singleton values were chosen to express the range of the EV3 DriveBase, from gentle to aggressive corrections, while remaining below the emergency turn speed of 45 deg/s.

The 15 rules are put in a 5×3 matrix, whose rows correspond to the five disturbance fuzzy sets and columns to the three trend of disturbance fuzzy sets:

Table 4.2: Fuzzy Rule-Based: rule matrix for lane keeping.

Disturbance \ Trend	**N**	**Z**	**P**
NL	PM	PL	PL
NS	PS	PS	PM
ZE	ZE	ZE	ZE
PS	NS	NS	NM
PL	NM	NL	NL

Listing 4.18: FuzzyRuleBasedAlgorithm rules tuple

```
"Rules": [
    (0, 0, 5),   # R1:  NL & N  -> PM
    (0, 1, 6),   # R2:  NL & Z  -> PL
    (0, 2, 6),   # R3:  NL & P  -> PL
    (1, 0, 4),   # R4:  NS & N  -> PS
    (1, 1, 4),   # R5:  NS & Z  -> PS
    (1, 2, 5),   # R6:  NS & P  -> PM
    (2, 0, 3),   # R7:  ZE & N  -> ZE
    (2, 1, 3),   # R8:  ZE & Z  -> ZE
    (2, 2, 3),   # R9:  ZE & P  -> ZE
    (3, 0, 2),   # R10: PS & N  -> NS
    (3, 1, 2),   # R11: PS & Z  -> NS
    (3, 2, 1),   # R12: PS & P  -> NM
    (4, 0, 1),   # R13: PL & N  -> NM
    (4, 1, 0),   # R14: PL & Z  -> NL
    (4, 2, 0),   # R15: PL & P  -> NL
],
```

The expanded rule table, encoded in the configuration as a list of tuples (disturbanceIndex, deltaIndex, outputIndex) and loaded into memory at the constructor express the IF-THEN rules and in the table below they are presented along with their physical meaning:

Table 4.3: Fuzzy Rule-Based: full IF-THEN rule set with physical meaning.

Rule	IF disturbance is	AND trend is	THEN turn rate is	Meaning
R1	NL	N	PM	Far left & recovering ⇒ moderate right turn
R2	NL	Z	PL	Far left & stable ⇒ strong right turn
R3	NL	P	PL	Far left & worsening ⇒ strong right turn
R4	NS	N	PS	Slightly left & recovering ⇒ gentle right turn
R5	NS	Z	PS	Slightly left & stable ⇒ gentle right turn
R6	NS	P	PM	Slightly left & worsening ⇒ moderate right turn
R7	ZE	N	ZE	Centered & recovering ⇒ go straight
R8	ZE	Z	ZE	Centered & stable ⇒ go straight
R9	ZE	P	ZE	Centered & worsening ⇒ go straight
R10	PS	N	NS	Slightly right & recovering ⇒ gentle left turn
R11	PS	Z	NS	Slightly right & stable ⇒ gentle left turn
R12	PS	P	NM	Slightly right & worsening ⇒ moderate left turn
R13	PL	N	NM	Far right & recovering ⇒ moderate left turn
R14	PL	Z	NL	Far right & stable ⇒ strong left turn
R15	PL	P	NL	Far right & worsening ⇒ strong left turn

The rule table has a symmetric structure with respect to 0: the output for (NL, Z) is PL (+50), while the output for (PL, Z) is NM (-50). It is equal in magnitude but opposite in sign. Similarly, (NS, Z) maps to PM (+30) and (PS, Z) maps to NM (-30). This symmetry ensures that the robot responds the same to left and right deviations.

The trend dictates the correction strength based on the situation being improved or worsened. When the trend is N (recovering) the magnitude of the correction is reduced and when the trend is P (worsening) the magnitude of the correction is increased. If the trend is Z, meaning the robot isn't recovering yet, then the magnitude of the correction remains increased.

The centered region, when the disturbance is ZE, maps every trend to ZE keeping the robot going straight regardless of previous changes in the disturbance.

Inference and Aggregation

During each control cycle, the fuzzification stage produces up to two non-zero membership values for the disturbance from adjacent sets and up to two non-zero values for the trend. This means that at any moment, 1, 2 or 4 rules can be simultaneously active, depending on whether both inputs are exactly on a set peak (1 rule), on a transition between two sets for one input and the other is on a set peak (2 rules) or both inputs are on transitions (4 rules).

Listing 4.19: FuzzyRuleBasedAlgorithm inference & aggregation

```
def inference(self, muDisturbance, muDelta):
    aggregation=[0.0]*len(self.outputSingletons)

    for disturbanceIndex, deltaIndex, outputIndex in self.rulesTable:
        ruleStrength=min(muDisturbance[disturbanceIndex], muDelta[
            deltaIndex])

        if ruleStrength>0.0:
            self.lastActiveRules.append((disturbanceIndex, deltaIndex,
                outputIndex, ruleStrength))

            if ruleStrength>aggregation[outputIndex]:
                aggregation[outputIndex]=ruleStrength

    return aggregation
```

The inference stage, implemented in the `inference()` method, firstly initializes the fuzzy output array with 0.0 activation degree for all singletons and then evaluates each of the 15 rules by computing the degree activation of the rule. The degree activation is computed by the fuzzy MIN operator, being the minimum of the two input membership values.

If one input has zero membership for a rule's premise, the rule doesn't activate and it's skipped. The method keeps track of the active rules and their strength in the `lastActiveRules` list for debugging.

In the case of multiple active rules map to the same singleton output, their strength

must be combined. The aggregation stage uses the fuzzy MAX operator in the way that for each output singleton, only the highest activation degree among all rules that activate it is kept. This is implemented as an array of the size corresponding to the number of singleton outputs and each entry holds the maximum degree activation for all the rules targeting that singleton.

Defuzzification

The defuzzification stage converts the aggregated fuzzy output into the crisp turn rate value. Since the output are represented as singletons, the defuzzification use center of mass formula:

$$\text{turn rate} = \frac{\sum_i \text{aggregation}[i] \cdot \text{singleton}[i]}{\sum_i \text{aggregation}[i]} \tag{4.3}$$

Listing 4.20: FuzzyRuleBasedAlgorithm defuzzification

```
def defuzzify(self, aggregation):
    numerator=0.0
    denominator=0.0

    for i in range(len(self.outputSingletons)):
        numerator+=aggregation[i]*self.outputSingletons[i]
        denominator+=aggregation[i]

    if denominator>0.0:
        return numerator/denominator
    else:
        return 0.0
```

The sum collects each output singleton. Singletons with zero aggregated strengths don't contribute to the turn rate and if no rules are active the default output is zero meaning driving straight.

Computing the Output

Listing 4.21: FuzzyRuleBasedAlgorithm compute method

```
def compute(self, setPoint, measuredPoint, dt):
    self.lastActiveRules=[]
    disturbance=setPoint-measuredPoint
    delta=self.updateDelta(disturbance)

    muDisturbance=self.fuzzifyDisturbance(disturbance)
    muDelta=self.fuzzifyDelta(delta)

    aggregation=self.inference(muDisturbance, muDelta)

    output=self.defuzzify(aggregation)

```

```
13      return disturbance, output
```

The `compute()` method combines all stages of the fuzzy logic to obtain the final turn rate the robot must execute. It receives the `setPoint` which is usually 0, indicating perfect balance between sensors, the `measuredPoint` which is the actual difference between sensors and the change in time.

The membership values of the disturbance and delta are retrieved from the `fuzzify()` methods. Based on them, the rules are evaluated to find out which are activated in the `inference()` method. The output turn rate is defuzzified from the aggregated fuzzy sets and is clamped so that the robot doesn't receive commands that isn't able to execute.

4.2 THE MAZE SOLVER MODULE

4.2.1 TASK DESCRIPTION

This project also implements a maze-solving algorithm in which the Lego Mindstorms EV3 navigates through a maze transposed as a matrix, being discovered at runtime. The maze is composed of cells separated by walls and the robot explores every possible cell while finding a path from the starting point to the finish. The robot uses four sensors to detect its environment: an ultrasonic sensor in the front to detect the obstacles ahead, two ultrasonic sensors on the left and on the right side to balance itself in corridors and to detect open passages, and a gyroscope sensor to maintain accurate heading after turns. The forward sensor is put on a motor that rotates 90 degrees left and right to allow the robot to find the possible paths when facing a wall in the front, if there are any.

The robot drives through the maze using a Depth First Search (DFS) algorithm that maintains a map of the explored paths in memory. The robot takes each cell as a node in a graph, records visited cells and which directions are open, then explores unvisited neighbors before backtracking. This guarantees complete exploration of any connected maze. The robot stops at each junction and scans for open passages in three directions: forward, left and right. It consults the map to determine which adjacent cells are unvisited and then advances into unvisited cells or backtracks along its path to find undiscovered branches. Between junctions, the robot uses control algorithms: PID, Fuzzy PI or Fuzzy Rule-Based to regulate its forward speed based on the distance to the next wall in the front.

The architecture of this application extends the same Object-Oriented, SOLID-compliant structure from the lane keeping application. The `initializeConfiguration()` method provides all necessary variables for the maze solver task.

Listing 4.22: MazeSolver task configuration

```
def initializeConfiguration(task, algorithm):
    elif task == "MazeSolver":
    taskConfiguration={
        "ForwardUltrasonicSensor": Port.S4,
        "LeftUltrasonicSensor":    Port.S1,
        "RightUltrasonicSensor":   Port.S2,
        "GyroSensor":              Port.S3,
```

```
        "CellDistanceNS":          278,
        "CellDistanceEW":          278,
        "OpenPassageThreshold":    90,
        "ObstacleThreshold":       30,
        "TargetForwardDistance":   60,
        "BaseSpeed":               90,
        "MinSpeed":                30,
        "TurnRate":                30,
        "HeadingThreshold":        3,
        "CenteringThreshold":      20,
        "CenteringGain":           0.5,
        "CorrectionClamp":         30,
        "LogInterval":             1,
    }
```

The three ultrasonic sensors and the gyroscope sensor are assigned to their respective ports, along with the motor for the forward sensor. `CellDistanceNS` and `CellDistanceEW` represent the length and width of each cell in the maze. `OpenPassageThreshold` is the minimum value the sideways sensors detect in order to consider it an open path. The `ObstacleThreshold` is the distance from the wall in front so that the robot stops and checks for alternative paths. `TargetForwardDistance` is the setpoint for the control of speed algorithm. `BaseSpeed` is the speed of the robot while driving in a long "corridor" from one cell to another, while `MinSpeed` is the minimum forward velocity to prevent the robot from taking too much time to traverse a cell. The `TurnRate` is the amount of degrees to turn in one second. The `HeadingThreshold` and `CenteringThreshold` define tolerance zones for errors smaller than 3 degrees from the fixed 90 degrees turn or 20 millimeters distance from the left or right wall. The `CenteringGain` is the conversion of the lateral disturbance ($leftDistance - rightDistance$) into a corrective turn angle, capped at ± 30 degrees to prevent oscillation.

4.2.2 THE MAZEMAP CLASS

Listing 4.23: MazeMap attributes

```
class MazeMap:
    NORTH = 0
    EAST  = 1
    SOUTH = 2
    WEST  = 3

    DX      = (0,  1,  0, -1)
    DY      = (1,  0, -1,  0)
    TURN    = (0, 90, 180, -90)
    DFS     = (0,  3,  1,  2)
    BACKDIR = {(0,1):0, (1,0):1, (0,-1):2, (-1,0):3}
```

The `MazeMap` class is the data structure that gives orientation and position to the robot in the maze. It keeps track of the explored cells of the maze and implements the

Depth-First Search navigation logic. The class uses a coordinate system based on a compass with cardinal directions encoded as integers. The `DX` and `DY` tuples define the displacement on the x and y axis when the robot changes its position. Moving North (forward) increases y by 1, moving East (right) increases x by 1, moving South (backward) decreases y by 1 and moving West (left) decreases x by 1. The `TURN` tuple contains the rotation angles in order to face each heading direction: 0 for forward, +90 for right, 180 for backward and -90 for left. The `DFS` list contains the priority order of the directions the robot takes and the `BACKDIR` dictionary provides the reverse lookup having as key the coordinate displacement and as value the corresponding direction.

Listing 4.24: MazeMap constructor

```
def __init__(self):
    self.visited           = set()
    self.path              = []
    self.x                 = 0
    self.y                 = 0
    self.heading           = 0
    self.expectedGyroAngle = 0
    self.cellWalls         = {}
    self.entryDirection    = None
```

The constructor initializes the starting position of the robot with the heading towards North, an empty set of visited cells and the empty path stack, which will store the sequence of cells the robot has passed through. With this, the robot can backtrack its steps once it reaches a dead end and avoid repeating the path over and over again.

Listing 4.25: MazeMap visit method

```
def markVisited(self):
    self.visited.add((self.x, self.y))
```

The `markVisited()` method adds the current cell to the visited cell. The robot will use this information to avoid going back to the same cell.

Listing 4.26: MazeMap gyro target

```
def gyroTarget(self):
    return self.expectedGyroAngle
```

The `gyroTarget()` method returns the expected gyroscope reading for the current heading. This is used to align the robot's heading after turns, since the physical turn may not be perfect and the gyroscope can have some drift.

Listing 4.27: MazeMap plan move method

```
def planMove(self, openAbsDirs):
    for step in self.DFS:
        absDir=(self.heading + step) % 4
```

```
4
5           if absDir not in openAbsDirs:
6               continue
7
8           nx=self.x + self.DX[absDir]
9           ny=self.y + self.DY[absDir]
10
11          if (nx, ny) not in self.visited:
12              return absDir, self.TURN[step]
13
14      return None, None
```

The `planMove()` method is the decision-making function to get the non-visited neighbouring cell. It receives as parameter the set of directions that are physically open (they are not walls) at the current junction. By iterating the DFS priority order, it checks for directions that are physically open and the cell in that direction wasn't visited. The first direction that satisfies both conditions is returned along with the required turn angle. If there aren't any unvisited and open directions, the method returns None and the backtracking method will be required.

The DFS priority order is defined as (0, 3, 1, 2) which translates to Forward, Left, Right and Backward. This means that the robot will continue straight until reaches a wall. It will turn left, if possible, otherwise will turn right. Finally, the robot turns around only if it's situated at a dead end.

Listing 4.28: MazeMap plan backtrack method

```
def planBacktrack(self):
    if not self.path:
        return None, None

    px, py=self.path[-1]
    backDir=self.BACKDIR[(px-self.x, py-self.y)]
    step=(backDir - self.heading) % 4

    return self.TURN[step], backDir
```

The `planBacktrack()` method handles the situation where the robot can't navigate to other unvisited cells. It looks at the top of the path stack to retrieve the previous cell, computes the direction needed return to using the `BACKDIR` dictionary and calculates the required turn angle. If the stack is empty, the method returns None, meaning the entire maze has been explored.

Listing 4.29: MazeMap apply turn method

```
def applyTurn(self, absDir):
    step = (absDir - self.heading) % 4
    self.expectedGyroAngle += self.TURN[step]
    self.heading = absDir
```

The `applyTurn()` method updates the heading before the physical turn. It calculates the turn step from the current heading to the new absolute direction and updates the expected gyroscope angle. This way the robot uses the gyroscope to align itself to the expected angle, ensuring accurate navigation through the maze.

Listing 4.30: MazeMap cell information

```
def storeCellInfo(self, openDirs):
        if(self.x, self.y) not in self.cellWalls:
            walls=0

            for direction in openDirs:
                walls |= (1 << direction)

            if self.entryDirection is not None:
                walls |= (1 << ((self.entryDirection + 2) % 4))

            self.cellWalls[(self.x, self.y)]=walls

            return walls
        else:
            return None
```

The `storeCellInfo()` method builds and updates the map of the maze inside the robot's memory. It uses bitmasking to encode all four cardinal directions in a single integer ranging from 0 to 15.

Firstly, the method checks its global map dictionary to see if the current cell was already stored. If it was, it returns None to avoid wasting time. Otherwise, it initializes the `walls` variable to 0. This variable serves as a 4-bit binary sequence representing the four directions: bit index 0 (value 1) = North, bit index 1 (value 2) = East, bit index 2 (value 4) = South, bit index 3 (value 8) = West.

The loop encodes the open directions by setting the corresponding bits to 1 using the left shift operator and aggregates them into the existing variable using bitwise OR. The bits of 1 represent an open path and the bits of 0 represent walls. Since the robot doesn't have a sensor in the rear, the `if` condition ensures that the robot came from an open path. `(self.entryDirection + 2)%4` returns the opposite direction of what the robot is heading to, wrapping to values between 0 and 3. For example, if it entered heading North(0), the opposite direction is South(2). It shifts 1 to that position, forcing to register it as an open path.

Finally, it stores the resulting integer in the `cellWalls` dictionary with the current coordinates as key. This way, the robot can retrieve the information about the cell and its walls by looking up the coordinates in the dictionary.

Listing 4.31: MazeMap commit move method

```
def commitMove(self):
    self.entryDirection=self.heading
    self.path.append((self.x, self.y))
```

```
4       self.x+=self.DX[self.heading]
5       self.y+=self.DY[self.heading]
6       self.markVisited()
```

The `commitMove()` method assigns the robot the current absolute direction to its entry direction in a cell, pushes the current position onto the path stack, sets the coordinates in the heading direction and marks the new cell as visited. This is called after the robot physically moves to the new cell, confirming that the move was successful and updating the state of the map.

Listing 4.32: MazeMap commit backtrack method

```
1 def commitBacktrack(self):
2     self.x, self.y=self.path.pop()
```

When backtracking, the `commitBacktrack()` method pops the previous position from the path stack to restore the coordinates where the robot came from. This is called after the robot physically moves back to the previous cell, confirming that the backtrack was successful and updating the state of the map.

4.2.3 THE MAZESOLVERCONTROLLER

The `MazeSolverController` class implements the `Controller` interface and executes the DFS algorithm for the robot to traverse the maze. It handles the sensors reading, motors turning, headings aligning, centering in corridors and precise driving for distances from cell to cell.

Listing 4.33: MazeSolverController constructor

```
1  def __init__(self, algorithm, configuration, streamer, ev3, robot, label
       ):
2      super().__init__(algorithm, configuration, streamer, ev3, robot,
           label)
3      self.forwardSensor      = UltrasonicSensor(configuration["
           ForwardUltrasonicSensor"])
4      self.leftSensor         = UltrasonicSensor(configuration["
           LeftUltrasonicSensor"])
5      self.rightSensor        = UltrasonicSensor(configuration["
           RightUltrasonicSensor"])
6      self.gyro               = GyroSensor(configuration["GyroSensor"])
7      self.cellDistanceNS     = configuration["CellDistanceNS"]
8      self.cellDistanceEW     = configuration["CellDistanceEW"]
9      self.openPassage            = configuration["OpenPassageThreshold"]
10     self.obstacleThreshold      = configuration["ObstacleThreshold"]
11     self.targetForwardDistance  = configuration["TargetForwardDistance"]
12     self.baseSpeed              = configuration["BaseSpeed"]
13     self.minSpeed               = configuration["MinSpeed"]
14     self.turnRate               = configuration["TurnRate"]
15     self.headingThreshold   = configuration["HeadingThreshold"]
16     self.centeringThreshold = configuration["CenteringThreshold"]
```

```
17      self.centeringGain      = configuration["CenteringGain"]
18      self.correctionClamp    = configuration["CorrectionClamp"]
19      self.logInterval        = configuration["LogInterval"]
20      self.mazeMap            = MazeMap()
```

The constructor takes the inherited attributes from the super abstract class `Controller` and adds the specific hardware for the maze: the three ultrasonic sensors, the gyroscope and the motor sensor. It instantiates the `MazeMap` object for the orientation and positioning of the robot and stores the configuration parameters for driving, turning and correction tasks.

Listing 4.34: MazeSolverController reading forward

```
1 def scanForward(self):
2     r0 = self.forwardSensor.distance()
3     wait(10)
4     r1 = self.forwardSensor.distance()
5     wait(10)
6     r2 = self.forwardSensor.distance()
7     r = [r0, r1, r2]
8     r.sort()
9     return r[1]
```

The `scanForward()` method reads the distance from the forward ultrasonic sensor. Due to the fact that ultrasonic sensors are susceptible to spike noise caused by the echoed signals bouncing off the angled walls, the method takes three consecutive readings, sorts them and returns the median. This way, spikes are ignored.

Listing 4.35: MazeSolverController reading every direction

```
1  def scanJunction(self):
2      openDirs = set()
3      h=self.mazeMap.heading
4
5      if self.scanForward()>=self.openPassage:
6          openDirs.add(h)
7      if self.leftSensor.distance()>=self.openPassage:
8          openDirs.add((h+3) % 4)
9      if self.rightSensor.distance()>=self.openPassage:
10         openDirs.add((h+1) % 4)
11
12     return openDirs
```

The `scanJunction()` method determines which directions are open in the current cell. It receives the values read by the sensors and compares them with the `openPassage` threshold. Each value that exceeds the threshold is added to the set of open directions. The method returns this set to be passed to the `MazeMap`'s `planMove()` method for decision making.

Listing 4.36: MazeSolverController gyro error computing

```
def gyroError(self):
        error = self.mazeMap.gyroTarget() - self.gyro.angle()
        return (error + 180) % 360 - 180
```

The `gyroError()` method is a helper method that computes the angular error between the expected gyro target and the current gyro reading. The error is normalized to the $[-180^o,\ 180^o]$ interval to ensure that the robot takes the shortest turn to align itself.

Listing 4.37: MazeSolverController heading aligning

```
def alignHeading(self):
    correction =self.gyroError()
    if abs(correction) > self.headingThreshold:
        self.robot.turn(correction)
        wait(20)
```

The `alignHeading()` method corrects the physical orientation of the robot to match its intended compass heading using the gyroscope. Mechanical errors such as wheel slippage can cause a deviation of the actual heading of the robot from the target, and after many turns, those accumulated errors will drive the robot into a wall. Hence, the need of this method to keep the robot from misaligning with the maze grid.

The method receives the error from the previous `gyroError()` method. If this needed correction is more than three degrees, then the robot executes a fixed turn of the computed angle and waits 20 ms for the mechanical system to settle. Otherwise, the alignment is skipped because such small corrections are within the sensor's noise.

Listing 4.38: MazeSolverController centering in corridor method

```
def centerInCorridor(self):
    leftDist = self.leftSensor.distance()
    rightDist = self.rightSensor.distance()

    if leftDist < self.openPassage and rightDist < self.openPassage and
       abs(leftDist - rightDist) > self.centeringThreshold:
        correction = (rightDist - leftDist) * self.centeringGain

        if correction > self.correctionClamp:
            correction = self.correctionClamp
        elif correction < -self.correctionClamp:
            correction = -self.correctionClamp

        self.robot.turn(correction)
        wait(10)
```

The `centerInCorridor()` method adjusts the lateral position of the robot within the corridor to prevent it from driving into a side wall, using the difference between the left and right distance from the walls. If both walls are there (there is no T junction) and the difference is relevantly high, the robot executes a small corrective turn. The correction angle is computed

as $difference \times centeringGain$, where this gain is 0.5 deg/mm, meaning a 100 mm offset produces a 50^o correction. The clamp acts as a protection so the robot doesn't turn to much.

Listing 4.39: MazeSolverController distance of a cell

```
def cellDist(self, absDir):
    if absDir%2==0:
        return self.cellDistanceNS
    else:
        return self.cellDistanceEW
```

The `cellDist()` method returns the physical distance the robot must traverse to cross one cell in the given absolute direction. Since the maze cells are rectangular, the distance depends on whether the robot is moving along the North-South axis or the East-West axis. For the North and South directions were used even indices and odd indices were used for the East and West directions.

Listing 4.40: MazeSolverControllerp turn executing

```
def executeTurn(self, degreeTurn):
    if degreeTurn == 0:
        return

    if abs(degreeTurn) == 90:
        startAngle = self.gyro.angle()
        targetAngle = startAngle + degreeTurn
        deadline = self.now() + 5.0

        turnRate=self.turnRate if degreeTurn > 0 else -self.turnRate
        self.robot.drive(0, turnRate)

        if degreeTurn > 0:
            while self.gyro.angle() < targetAngle and self.now() <
                deadline:
                wait(10)
        else:
            while self.gyro.angle() > targetAngle and self.now() <
                deadline:
                wait(10)
        self.robot.stop()
    else:
        self.robot.turn(degreeTurn)
```

The `executeTurn()` method performs the physical rotation of the robot by handling three cases:

The first case, the `degreeTurn` is 0^o. This means that the robot is already facing the correct direction and no action is needed.

The second case, the `degreeTurn` is $\pm 90^o$. The method reads the starting angle from the gyro, computes the target angle and rotates with 0 linear velocity. A tight while loop continuously check the gyro sensor at a 10 ms interval until the targt heading is reached or a

deadline of 5 seconds is exceeded.

The third case treats turn-arounds where the method uses the DriveBase's `robot.turn()` method which uses the motor encoders rather than the gyro to rotate to robot faster.

Listing 4.41: MazeSolverController cell driving

```
def driveCell(self, absDir):
        cellTarget = self.cellDist(absDir)
        travelDist = 0.0
        prevSpeed  = 0.0
        lastTime   = self.now()
        self.algorithm.reset()
        wait(20)

        forward = self.scanForward()

        while travelDist < cellTarget and forward >= self.
            obstacleThreshold:
            currentTime = self.now()
            dt          = currentTime - lastTime
            lastTime    = currentTime

            forward = self.scanForward()

            if forward < self.obstacleThreshold:
                break

            disturbance, speed = self.algorithm.compute(self.
                targetForwardDistance, forward, dt)
            if speed < self.minSpeed:
                speed = self.minSpeed
            if speed > self.baseSpeed:
                speed = self.baseSpeed

            self.robot.drive(speed, 0)

            travelDist += 0.5 * (prevSpeed + speed) * dt
            prevSpeed   = speed

            self.nextStep()
            self.streamer.send(self.step, disturbance, speed)
            self.logger.log(self.step, disturbance, speed)

            if self.step % self.logInterval == 0:
                print("Step "+str(self.step)+" Forward: "+str(forward)+"
                    mm Speed: "+str(speed)+" mm/s")
                if isinstance(self.algorithm, FuzzyRuleBased):
                    print(self.algorithm.logRules())
            wait(10)

```

```
42          self.robot.stop()
```

The `driveCell()` method drives the robot from one cell to another using the selected control algorithm: PID, Fuzzy PI or Fuzzy Rule-Based to regulate the speed to drive forward. This is where the same algorithm classes from the lane keeping application are reused, but with the fundamental change in the algorithm's output: for the lane keeping, the algorithm controls the turn rate to stay centered, while for the maze solver, the algorithm controls the forward speed approach the next wall safely.

At each control cycle, the forward ultrasonic sensor is read and if its value is smaller than the obstacleThreshold, the robot stops. Otherwise, the distance is passed to the algorithm's `compute()` method. When the wall is far away, the algorithm produces a high speed, up to the base speed. As the wall gets closer to the target distance, the output speed decreases towards the minimum speed.

The distance travelled is tracked using the velocity law: at each loop iteration, the distance increment is computed as the average between the previous and current speed multiplied with the change in time. When the integrated distance reaches the cell dimension, the robot stops and the control returns to the main DFS loop.

The algorithm is reset at the beginning of each cell drive to clear integral accumulator and previous disturbance.

4.2.4 MAIN EXECUTION LOOP

Listing 4.42: MazeSolverController move executing

```
def executeMove(self, degreeTurn, direction, commit):
    self.executeTurn(degreeTurn)
    wait(20)
    self.mazeMap.applyTurn(direction)
    self.alignHeading()
    self.centerInCorridor()
    self.driveCell(direction)
    commit()
```

This helper method combines the low-level physical maneuvers of turning, aligning, centering and driving with structural map updates into a single high-level move execution. It receives the turn angle, the absolute direction to move and the commit function to update the map after the physical move is completed. This method is called for both forward moves and backtracking moves, since they share the same physical execution sequence.

Listing 4.43: MazeSolverController run method

```
def run(self):
    self.printHeader(self.algorithmLabel())
    self.gyro.reset_angle(0)
    self.mazeMap.markVisited()

    try:
```

```
        while True:
            openDirs=self.scanJunction()
            print("Cell ("+str(self.mazeMap.x)+","+str(self.mazeMap.y)+"
                )"+" H="+str(self.mazeMap.heading)+" Open="+str(openDirs)
                )

            walls=self.mazeMap.storeCellInfo(openDirs)
            if walls is None:
                walls=self.mazeMap.cellWalls.get((self.mazeMap.x, self.
                    mazeMap.y), 0)
            self.streamer.sendCell(self.mazeMap.x, self.mazeMap.y, walls
                )

            absDir, degreeTurn=self.mazeMap.planMove(openDirs)

            if absDir is None:
                degreeTurn, backDir = self.mazeMap.planBacktrack()

                if degreeTurn is None:
                    print("Maze fully explored")
                    self.ev3.speaker.beep()
                    break

                print("Backtrack: turn "+str(degreeTurn)+" deg")

                self.executeMove(degreeTurn, backDir, self.mazeMap.
                    commitBacktrack)
            else:
                print("Move: dir="+str(absDir)+" turn="+str(degreeTurn)+
                    " deg")

                self.executeMove(degreeTurn, absDir, self.mazeMap.
                    commitMove)

            self.nextStep()

    except KeyboardInterrupt:
        self.robot.stop()
        self.ev3.speaker.beep()
        print("\nMAZE SOLVER STOPPED at ("+str(self.mazeMap.x)+","+str(
            self.mazeMap.y)+")\n")
    finally:
        self.streamer.close()
        self.logger.close()
```

The `run()` method is the main execution loop that orchestrates the complete maze exploration. It combines the abstract graph traversal of the MazeMap with the physical execution of the controller. The method begins with identifying the algorithm, printing the header with the maze dimensions and threshold values, resetting the sensor motor and gyro to their zero positions and marking the the starting cell as visited.

Then the robot scans the current junction to read forward, left and right sensors and

returns the set of open directions. The open directions are passed to `storeCellInfo()` to obtain information about the current cell and pass the resulting mask to the host PC via `WifiStreamer`.

The `planMove()` method is then called to determine the next move. If an unvisited path is found, it reurns the absolute direction and the steering rotation. The controller executes the move by calling the `executeMove()` method, passing the `self.mazeMap.commitMove` function to append the current coordinate position onto the history tracking stack.

If no unvisited direction is available, `planMove()` returns `None`. This triggers the backtrack sub-routine, invoking `planBacktrack()` to peek at the top element in the stack. It determines the rotation angle needed to step backward to its parent cell and calls `executeMove()` with the `self.mazeMap.commitBacktrack`.

If the stack is empty, `planBacktrack()` returns `None`, which signals that the entire maze has been explored and the loop can be exited.

The loop is wrapped in a try-except block to interrupt from keyboard to stop the robot and close the WiFi stream.

Fuzzy Rules for Maze Solver

Since the `MazeSolverController` uses the same `Algorithm` interface as the `LaneKeepingController`, the same algorithms can be reused with a different interpretation of the input and output. The disturbance is still the error between the target distance and the forward sensor reading, but now the output is interpreted as a forward speed rather than a turn rate.

As such, the same PID and Fuzzy PI algorithms can be reused without modification. However, the Fuzzy Rule-Based algorithm was redesigned with a new set of rules to better suit the speed control task. The new rules are based on the distance to the wall in front and are designed to produce a smooth deceleration as the robot approaches the target distance, while maintaining a high speed when the wall is far away.

The rules are defined as follows:

These five rules are designed to take into consideration only the distance to the wall in front, since the trend of the error is not relevant for this task. The rules produce a high speed when the robot is far from the target distance. As the robot gets closer to the target, the speed decreases to medium and then low, ensuring a smooth approach to the wall without overshooting.

The rules are implemented in the `FuzzyRuleBased` class, which uses the same fuzzification and defuzzification methods as the lane keeping version, but with the new rule set defined above. Those methods worked with two premises so these rules take the second premise the same for all to act as a neutral element.

Table 4.4: Fuzzy Rule-Based: full IF-THEN rule set with physical meaning.

Rule	IF disturbance is	AND trend is	THEN speed is	Meaning
R1	NL	Z	PL	Far from target and far from wall& don't care ⇒ high speed
R2	NS	Z	PL	Close to target and far from wall& don't care ⇒ high speed
R3	ZE	Z	PM	To the target and far from wall& don't care ⇒ medium speed
R4	PS	Z	PS	Close to target and far from wall & don't care ⇒ low speed
R5	PL	Z	PS	Far from target and close to wall & stable ⇒ low speed

4.3 THE TELEMETRY INTERFACE

4.3.1 ROBOT-SIDE COMPONENT: WIFISTREAMER

To handle the telemetry data streaming from the robot to the host PC, the class`WifiStreamer` was implemented. This class initializes a network socket using User Datagram Protocol (UDP).

The UDP protocol was chosen over TCP because it trasmits packets without waiting for confirmation from the receiver, which is suitable for live telemetry data where low latency is more important than guaranteed delivery.

Listing 4.44: WifiStreamer constructor

```
def __init__(self, configuration, label):
    self.ip=configuration["PC_IP"]
    self.port=configuration["PC_PORT"]
    self.label=label
    self.socket=None

    if configuration["EnableStreaming"]:
        try:
            self.socket=socket.socket(socket.AF_INET, socket.SOCK_DGRAM)
            self.sendMeta()
            print("Wifi streaming enabled-sending to " + self.ip + ":" +
                str(self.port))
        except Exception as error:
            print("Warning: Could not create socket-" + str(error))
```

```
14                self.socket=None
```

The constructor initializes the network destination and the UDP socket is streaming is enabled. It takes the IP address and port number from the configuration and sends an initial metadata packet to the PC to identify the source of the telemetry data. If the socket creation fails, it catches the exception and disables streaming.

Listing 4.45: WifiStreamer serializer

```
1 def sendPacket(self, packet):
2     if self.socket is None:
3         return
4     try:
5         self.socket.sendto(packet.encode(), (self.ip, self.port))
6     except Exception as error:
7         print("Warning: Could not send data-" + str(error))
```

The `sendPacket()` method encodes the telemetry data into a string format and sends it to the PC using the UDP socket. Since Python socket methods can't send pure strings, they serializes alphanumeric strings into UTF-8 byte arrays and this method is the base wrapper that pushes the data onto the network socket layer.

Listing 4.46: WifiStreamer algorithm data

```
1 def send(self, step, disturbance, output):
2     self.sendPacket(str(step)+","+str(disturbance)+","+str(output)+"\n")
```

The `send()` method is a helper function that formats the step, disturbance and output, either turn rate or speed, from the `compute()` methods into a comma-separated string with a newline character at the end. This format is parsed on the PC side to be analyzed later.

Listing 4.47: WifiStreamer algorithm data

```
1 def send(self, step, disturbance, output):
2     self.sendPacket(str(step)+","+str(disturbance)+","+str(output)+"\n")
```

The `sendCell()` method formats the cell coordinates and wall information into a comma-separated string with a newline character at the end. This method is called by the `MazeSolver` to send the explored cell information to the PC for visualization.

4.3.2 PC-SIDE COMPONENT: TELEMETRYVIEWER

Graph Window

The `GraphWindow` class creates a matplotlib figure in a tkinter window using the `FigureCanvasTkAgg` class. It displays two animated lines: a red one which represents the disturbance and the input for the algorithm's `compute()` method and a blue line which represents the output of the algorithm.

Listing 4.48: Graph Window

```
def buildFigure(self, task: str, algorithm: str):
    title, inLabel, outLabel=makeLabels(task, algorithm)

    self.title(title)

    self.figure, self.axis=matplotlib.pyplot.subplots(figsize=(10, 5))
    self.figure.patch.set_facecolor("#1e1e2e")
    self.axis.set_facecolor("#2a2a3e")

    self.lineDisturbance, = self.axis.plot(
        [], [], color=COLOUR_RED, linewidth=2, label=inLabel
    )

    self.lineOutput, = self.axis.plot(
        [], [], color=COLOUR_BUTTON_ACTIVE, linewidth=2, label=outLabel
    )

    self.axis.set_xlabel("Step", color=COLOUR_TEXT, fontsize=11,
       fontweight="bold")
    self.axis.set_ylabel("Value", color=COLOUR_TEXT, fontsize=11,
       fontweight="bold")
    self.axis.set_title(title, color=COLOUR_TEXT, fontsize=13,
       fontweight="bold")
    self.axis.tick_params(colors=COLOUR_SUBTEXT)
    self.axis.spines[:].set_color(COLOUR_BORDER)
    self.axis.set_xlim(0, 100)
    self.axis.set_ylim(-100, 100)
    self.axis.grid(True, alpha=0.2, linestyle="--", color=COLOUR_BORDER)

    self.axis.legend(
        loc="upper right", fontsize=9,
        facecolor=COLOUR_SURFACE, edgecolor=COLOUR_BORDER,
        labelcolor=COLOUR_TEXT,
    )

    self.canvas = FigureCanvasTkAgg(self.figure, master=self)
    self.canvas.get_tk_widget().pack(fill=tkinter.BOTH, expand=True,
       padx=8, pady=8)

    toolbar = NavigationToolbar2Tk(self.canvas, self)
    toolbar.update()
    toolbar.pack(side=tkinter.BOTTOM, fill=tkinter.X)
```

The animation runs at 50 ms interval, having each frame copying the current data from the robot via the `WifiStreamer` and updating the line objects. The graph always shows the line graph regardless of which task is running, since the labels are automaticallu adjusted through the `makeLabels()` function to reflect the current task and algorithm.

Maze Map Window

The `MazeMapWindow` class creates a second and optional window to visualize the explored maze map. It receives the cell information from the robot via the `WifiStreamer` and draws the cells with their walls on the canvas. The current position of the robot is highlighted in blue, the start cell in green, and all other cells in grey. Walls are drawn as white lines on the edge of each cell, using the bitmask from the CELL packets: a set bit means open passage, while a cleared bit means a wall.

Listing 4.49: Maze Map Window

```
def drawMazeCell(self, x, y, walls, isCurrent, isStart):
    fill=COLOUR_BUTTON_ACTIVE if isCurrent else (COLOUR_GREEN if isStart
        else COLOUR_SURFACE)
    self.axis.add_patch(Rectangle((x, y), 1, 1, facecolor=fill,
       edgecolor=COLOUR_BORDER, linewidth=1, zorder=1))
    textColor=COLOUR_BACKGROUND if (isCurrent or isStart) else
       COLOUR_TEXT
    self.axis.text(x+0.5, y+0.5, f"({x},{y})", color=textColor, fontsize
       =6, ha="center", va="center", zorder=3)

    W=COLOUR_TEXT
    lw=2

    if not (walls & 1):
        self.axis.add_line(Line2D([x, x+1], [y+1, y+1], color=W,
           linewidth=lw, zorder=2))
    if not (walls & 2):
        self.axis.add_line(Line2D([x+1, x+1], [y, y+1], color=W,
           linewidth=lw, zorder=2))
    if not (walls & 4):
        self.axis.add_line(Line2D([x, x+1], [y, y], color=W, linewidth=
           lw, zorder=2))
    if not (walls & 8):
        self.axis.add_line(Line2D([x, x], [y, y+1], color=W, linewidth=
           lw, zorder=2))
```

The animation updates at 300 ns interval to reduce CPU usage, since the maze map changes only once per cell transition.

4.3.3 THE APPLICATION LAUNCHER

The `LauncherApp` class is the main entry point of the PC application. It creates the interface for selecting the task (Lane Keeping or Maze Solver) and the algorithm (PID, Fuzzy PI or Fuzzy Rule-Based), the EV3 hostname and the maze map checkbox.

When the user clicks "Start Robot", the launcher performs three actions in sequence:

Firstly, the method `patchRobotFile()` modifies the two global variables TASK and ALGORITHM in the `fuzzy_robot.py` file using regular expressions and patches the `PC_IP` address to the current PC's current local IP using a dummy UDP socket.

Secondly, the graph window and optionally the maze map window are opened.

Finally, the method `launchEV3()` starts a background thread that uses SCP to copy the patched robot file to the EV3 and SSH to execute it via brickrun. The deployment runs in a Powershell subprocess with a new console window so that the user can see the robot's console output live.

4.4 OBJECT-ORIENTED ARCHITECTURE

4.4.1 CLASS HIERARCHY

The software architecture of this project is organized around two parallel class hierarchies connected by composition.

The `Algorithm` abstract base class defines the interface for the control strategies.

Listing 4.50: Algorithm Base Class

```
class Algorithm:
    def compute(self, setpoint, measuredpoint, dt):
        raise NotImplementedError

    def reset(self):
        raise NotImplementedError
```

It define the `compute()` method that takes the setpoint, measured point and time delta as input and returns the control output and the `reset()` method to clear any internal state of the algorithm when starting a new control sequence.

Three concrete subclasses implement this interface:

- `PIDAlgorithm`: Implements a standard Proportional-Integral-Derivative controller.
- `FuzzyPIAlgorithm`: Implements a Fuzzy PI controller with three membership functions and three rules to scale the gains adaptively.
- `FuzzyRuleBased`: Implements a pure Mamdani Fuzzy Logic controller with 15 rules designed for the specific task.

The `Controller` abstract base class defines the interface for the logic execution of a specific task. Both tasks share the algorithm instance, the WiFi streamer, the CSV logger, the EV3 hardware references, the stopwatch, the step counter and the `algorithmLabel()` method.

Listing 4.51: Controller Base Class

```
class Controller:
    def __init__(self, algorithm, configuration, streamer, ev3, robot,
        label):
        self.algorithm=algorithm
        self.configuration=configuration
        self.streamer=streamer
```

```
        self.ev3=ev3
        self.robot=robot
        self.stopwatch=StopWatch()
        self.step=0
        self.logger=FileLogger(label)

    def run(self):
        raise NotImplementedError

    def now(self):
        return self.stopwatch.time() / 1000.0

    def nextStep(self):
        self.step+=1
        return self.step, self.now()

    def algorithmLabel(self):
        if isinstance(self.algorithm, FuzzyPIAlgorithm):
            return "FuzzyPI"
        elif isinstance(self.algorithm, FuzzyRuleBased):
            return "FuzzyRuleBased"
        else:
            return "PID"
```

Two concrete subclasses implement the `run()` method:

- `LaneKeepingController`: Implements the lane keeping logic, reading the left and right sensors to compute the error and using the algorithm to compute the turn rate.
- `MazeSolverController`: Implements the maze solving logic, using the `MazeMap` class to keep track of the explored maze and making decisions based on the open paths and visited cells. It uses the algorithm to compute the forward speed based on its distance from a wall.

These two hierarchies are connected by composition. Each Controller has an Algorithm instance passed through the constructor.

4.4.2 SOLID PRINCIPLES

- Single Responsibility Principle: Each class has one reason to change: `FuzzyRuleBased` computes the output based on fuzzy logic applied on rules, `MazeMap` manages graph traversal and mapping, `WifiStreamer` handles network communication and `FileLogger` handles data logging. No class mixes concerns.
- Open/Closed Principle: New algorithms can be added by creating a new subclass of `Algorithm` without modifying the existing code. The `FuzzyRuleBased` class was added without changing the `PIDAlgorithm` or `FuzzyPIAlgorithm` classes, and the controllers use it. Similarly, new tasks can be added as new subclasses of the `Controller` without modifying the algorithm classes.

- Liskov Substitution Principle: Any `Algorithm` can replace any other without breaking the controller, because they all satisfy the same `compute(setpoint, measuredPoint, dt) -> (disturbance, output)`. The maze solver passes the target distance and the measured distance, while the lane keeper passes 0 as set point and the sensor difference and measured point. Both produce a (disturbance, output) tuple that the controller uses without knowing which algorithm generated it.
- Interface Segregation Principle: The `Algorithm` interface exposes only `compute()` and `reset()`. It doesn't force all algorithms to implement fuzzy inference or specific methods for the PID. The `FuzzyRuleBased` class has additional methods (`fuzzifyDelta`, `inference()` and `defuzzification()`) that are internal implemenation details and not part of the public interface.
- Dependency Inversion Principle: The controllers depend on the abstract `Algorithm` interface rather than concrete implementations. The concrete algorithm is selected at startup by the factory method `createController()` and injected into the controller constructor. The `Controller` base class receives its label as a parameter rather than reading global variables, ensuring that all dependencies flow inward through the constructor.

4.4.3 FACTORY PATTERN

Listing 4.52: Controller Factory

```
class Controller:
    def createController(task, algorithmName):
    configuration=initializeConfiguration(task, algorithmName)
    ev3, robot=hardware_setup(configuration)
    label=task+"_"+algorithmName
    streamer=WifiStreamer(configuration, label)

    if algorithmName=="PID":
        algorithm=PIDAlgorithm(configuration)
    elif algorithmName=="FuzzyPI":
        algorithm=FuzzyPIAlgorithm(configuration)
    elif algorithmName=="FuzzyRuleBased":
        algorithm=FuzzyRuleBased(configuration)
    else:
        raise ValueError("Unknown algorithm: "+algorithmName)

    if task=="LaneKeeping":
        return LaneKeepingController(algorithm, configuration, streamer,
            ev3, robot, label)
    elif task=="MazeSolver":
        return MazeSolverController(algorithm, configuration, streamer,
           ev3, robot, label)
    else:
        raise ValueError("Unknown task: "+task)
```

The `createController()` method serves as a factory for creating instances of the controllers and algorithms based on the selected task and algorithm name.

4.4.4 STRATEGY PATTERN

5. EXPERIMENTAL RESULTS

5.1 EXPERIMENTAL SETUP

5.2 LANE KEEPING RESULTS

5.2.1 DISTURBANCE OVER TIME

5.2.2 PERFORMANCE COMPARISON

5.2.3 TURN RATE OUTPUT

5.2.4 RULE ACTIVATION ANALYSIS

5.3 MAZE SOLVER RESULTS

5.3.1 EXPLORATION MAP

5.3.2 SPEED REGULATION

5.3.3 MAZE SOLVER PERFORMANCE

6. CONCLUSIONS AND PERSPECTIVES

6.1 CONCLUSIONS

6.2 SYNTHESIS OF CONTRIBUTIONS

The main contributions of this project are:

6.3 PERSPECTIVES FOR DEVELOPMENT

Several directions for future development arise from this work:

DECLARAȚIE DE AUTENTICITATE A LUCRĂRII DE FINALIZARE A STUDIILOR *

Subsemnatul **ION POPESCU**, legitimat cu **CI** seria **TZ** nr. **100772**, CNP **5000102355779**, autorul lucrării **TITLUL LUCRĂRII DE FINALIZARE A STUDIILOR** elaborată în vederea susținerii examenului de finalizare a studiilor de **LICENȚĂ** organizat de către Facultatea **AUTOMATICĂ ȘI CALCULATOARE** din cadrul Universității Politehnica Timişoara, sesiunea **IUNIE** a anului universitar **2022**, coordonator **dr.ing. MIHAI IONESCU**, luând în considerare conținutul art. 34 din *Regulamentul privind organizarea și desfășurarea examenelor de licență/diplomă și disertație*, aprobat prin HS nr. 109/14.05.2020 și cunoscând faptul că în cazul constatării ulterioare a unor declarații false, voi suporta sancțiunea administrativă prevăzută de art. 146 din Legea nr. 1/2011 – legea educației naționale și anume anularea diplomei de studii, declar pe proprie răspundere, că:

- această lucrare este rezultatul propriei activități intelectuale,
- lucrarea nu conține texte, date sau elemente de grafică din alte lucrări sau din alte surse fără ca acestea să nu fie citate, inclusiv situația în care sursa o reprezintă o altă lucrare/alte lucrări ale subsemnatului.
- sursele bibliografice au fost folosite cu respectarea legislației române şi a convențiilor internaționale privind drepturile de autor.
- această lucrare nu a mai fost prezentată în fața unei alte comisii de examen de licență/diplomă/disertație.

Timișoara,

Data Semnătura

_____ _____

* Declarația se completează „de mână” şi se inserează în lucrarea de finalizare a studiilor, la sfârşitul acesteia, ca parte integrantă.